

Cooperation models

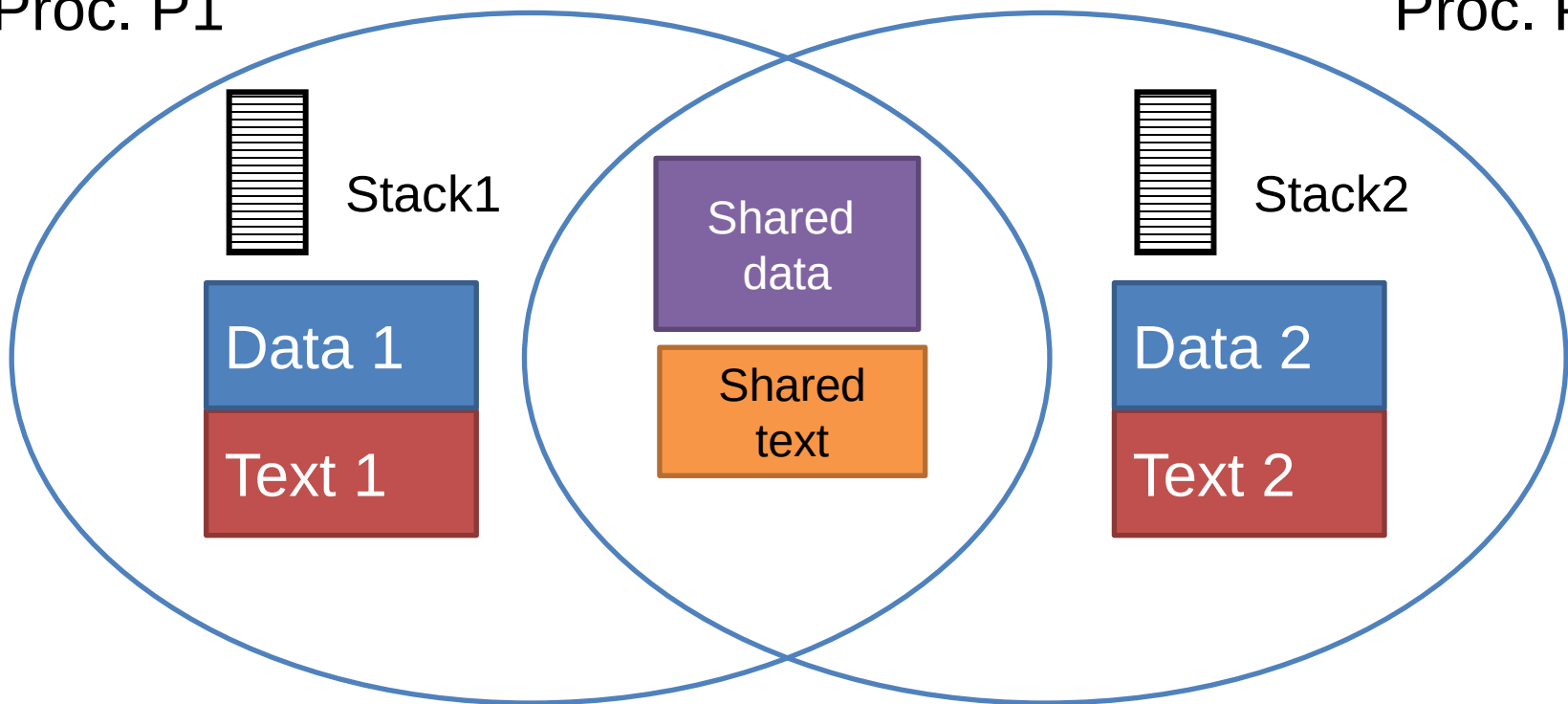
Global vs. local model

- Global environment
 - Processes/threads can share data
 - Cooperation via ***shared memory variables***
- Local environment
 - processes/threads do not share data
 - No shared memory
 - Cooperation via ***explicit messages***

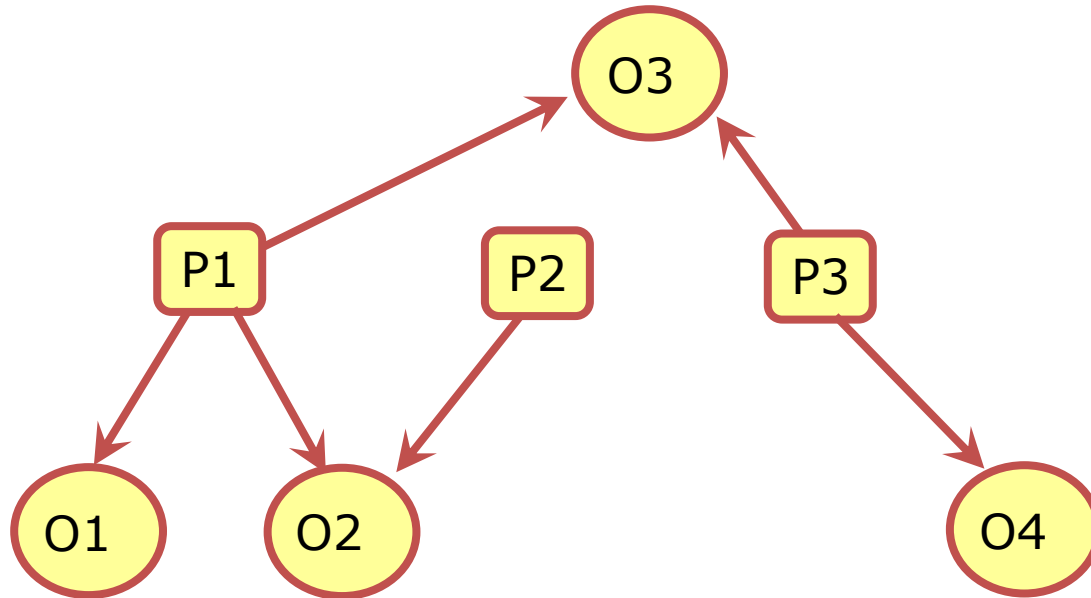
Global environment

Proc. P1

Proc. P2



Global environment

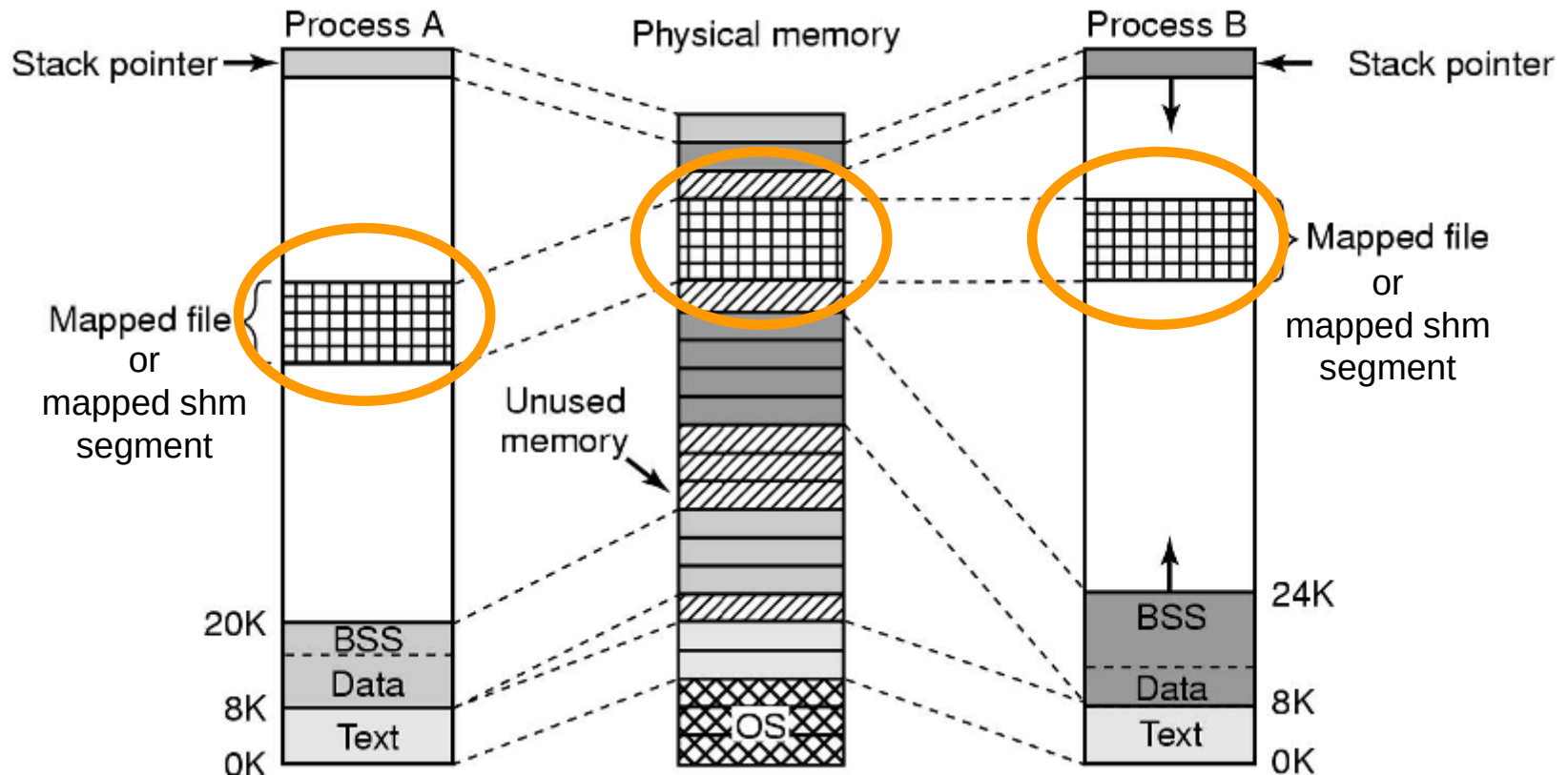


O1, O4 private objects

O2, O3 shared objects

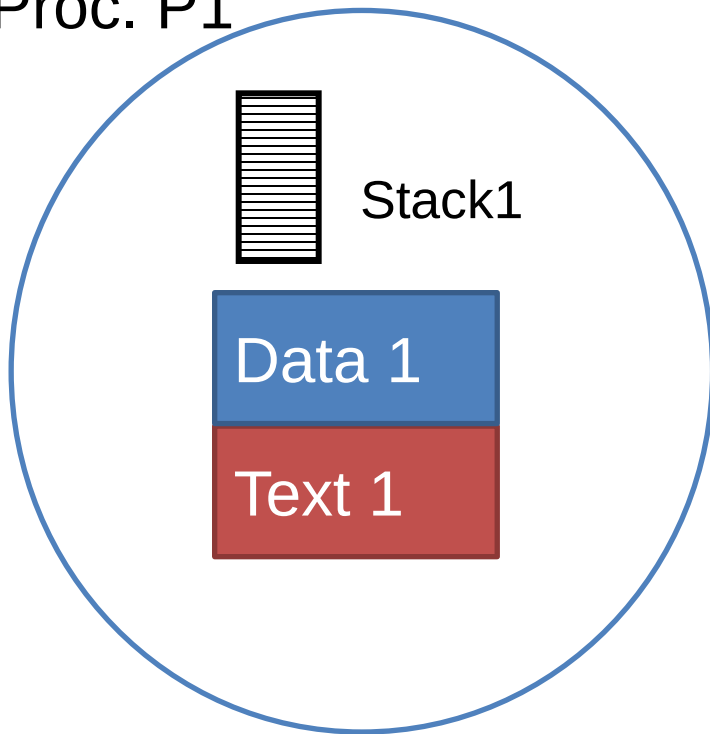
- competition, cooperation

Segment sharing using memory mapping in Unix

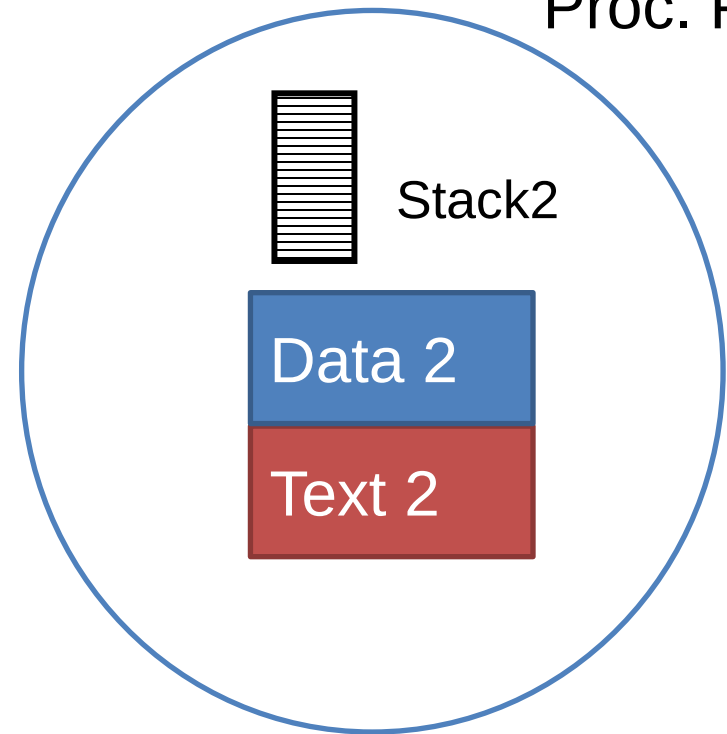


Local environment

Proc. P1



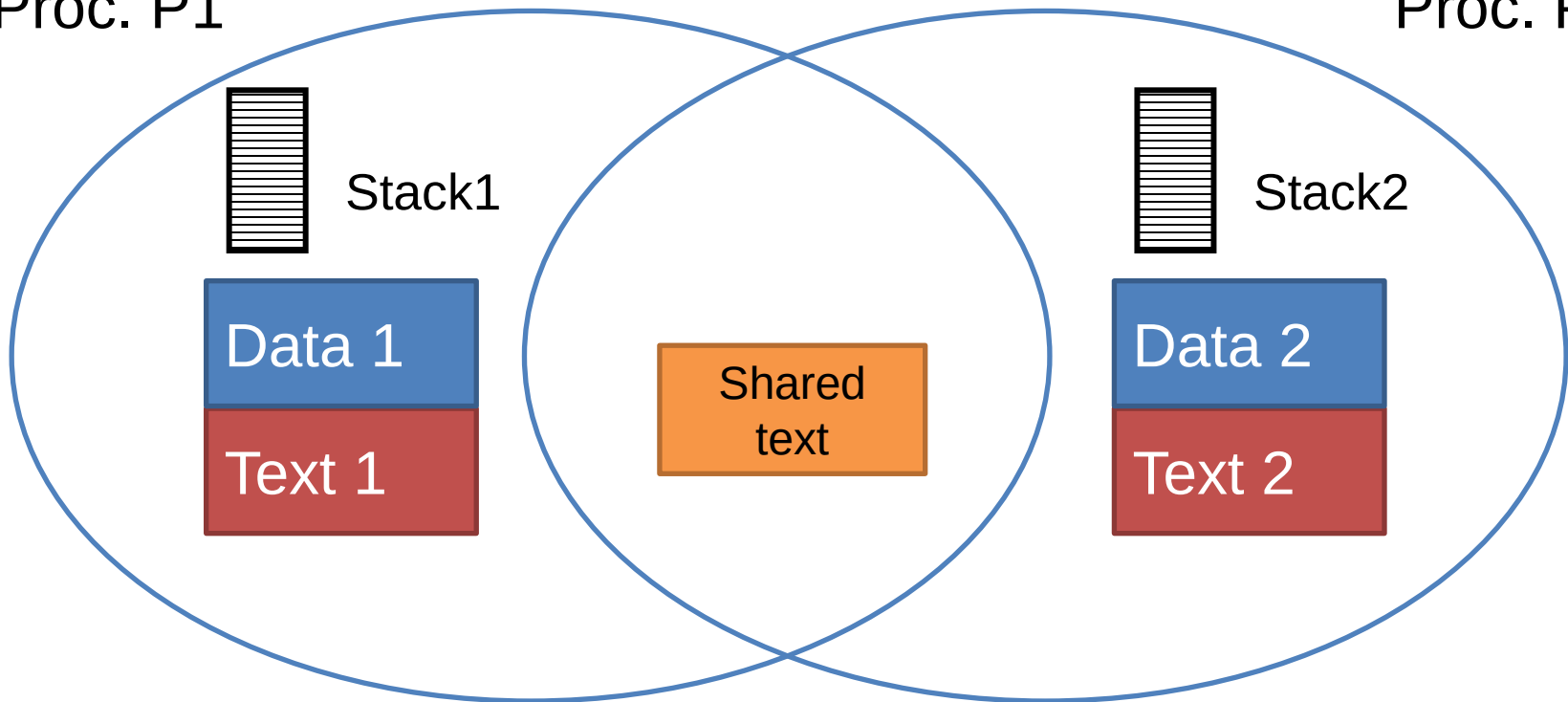
Proc. P2



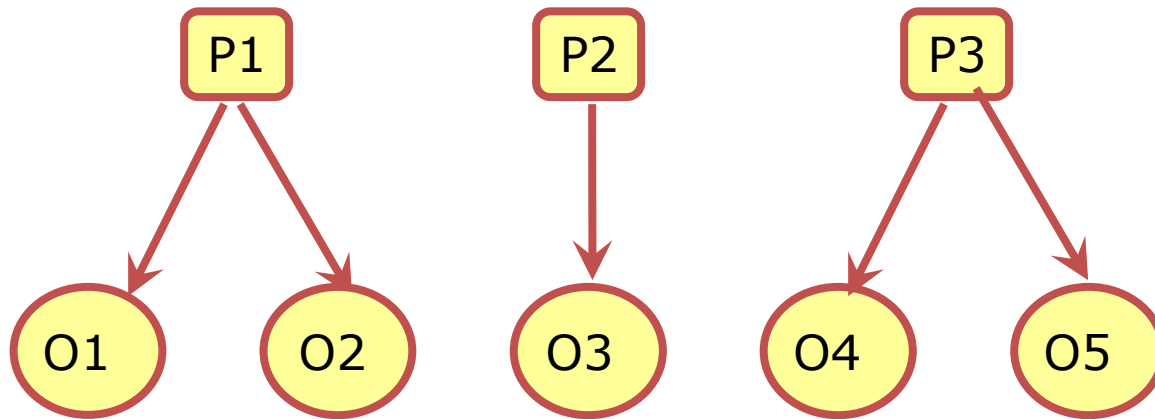
Local environment

Proc. P1

Proc. P2



Local environment

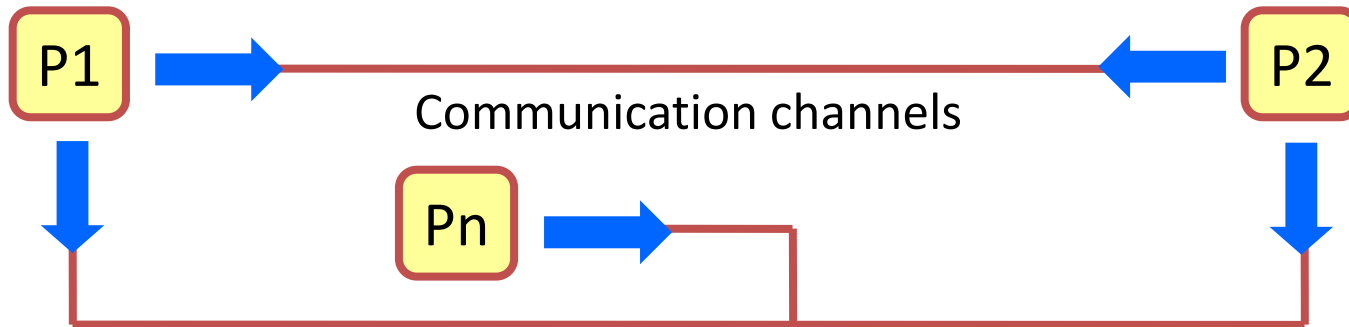


O1-O5 are private objects

Competition through server processes

Cooperation through explicit communication

Local environment



Cooperation (communication, synchronization) by means of message passing

Synchronization

Synchronization Motivation

- When threads concurrently read/write shared memory, program behavior is undefined
 - Two threads write to the same variable; which one should win?
- Thread schedule is non-deterministic
 - Behavior changes when re-run program
- Compiler/hardware instruction reordering
- Multi-word operations are not atomic

Can this panic?

NOTE: all functions do not have side effects, i.e., they are stateless!

Thread 1

```
p = someFn();  
isInitialized = true;
```

Thread 2

```
while (! isInitialized ) ;  
q = aFn(p);
```

```
if ( q != aFn(someFn() ) )  
    panic
```

Can this panic?

NOTE: all functions do not have side effects, i.e., they are stateless!

Thread 1

```
p = someFn();  
isInitialized = true;
```

Thread 2

```
while (! isInitialized ) ;  
q = aFn(p);
```

```
if ( q != aFn(someFn()) )  
    panic
```

Yes, it can panic!!

Both the compiler **and** processor may reorder operations!!

Why Reordering?

- Why do compilers reorder instructions?
 - Efficient code generation requires analyzing control/data dependency
 - If variables can spontaneously change, most compiler optimizations become impossible
- Why do CPUs reorder instructions?
 - **Write buffering**: allow next instruction to execute while write is being completed
- How to fix it? **Memory barrier** instructions
 - Instruction to compiler/CPU
 - All ops before barrier complete before barrier returns
 - No op after barrier starts until barrier returns
 - **NOTE**: synchronizations (e.g., locks) introduce implicit memory barrier!

Too Much Milk Example

	Person A	Person B
12:30	Look in fridge. Out of milk.	
12:35	Leave for store.	
12:40	Arrive at store.	Look in fridge. Out of milk.
12:45	Buy milk.	Leave for store.
12:50	Arrive home, put milk away.	Arrive at store.
12:55		Buy milk.
1:00		Arrive home, put milk away. Oh no!

Definitions

- **Race condition** : output of a concurrent program depends on the order of operations between threads (ops interleaving)
- **Mutual exclusion**: only one thread does a particular thing at a time
 - **Critical section**: piece of code that only one thread can execute at once
- **Lock**: Synchronization variable that provides mutual exclusion, preventing someone from doing something
 - Lock before entering critical section, i.e., before accessing shared data
 - Unlock when leaving, after done accessing shared data
 - wait if locked (**all synchronizations involves waiting either active or passive waiting!!**)

Too Much Milk, Try #1

- Correctness property
 - Someone buys if needed (*liveness*)
 - At most one person buys (*safety*)
- Try #1: leave a note...

Too Much Milk, Try #1

Thread A

```
if (!note) {  
    if (!milk)  
        leave note  
    buy milk  
    remove note  
}
```

Thread B

```
if (!note){  
    if (!milk)  
        leave note  
    buy milk  
    remove note  
}
```

Too Much Milk, Try #2

Thread A

leave note A

```
if (!note B) {
```

```
    if (!milk)
```

```
        buy milk
```

```
}
```

remove note A

Thread B

leave note B

```
if (!noteA){
```

```
    if (!milk)
```

```
        buy milk
```

```
}
```

remove note B

Too Much Milk, Try #3

Thread A

```
leave note A
while (note B) // X
    do nothing;
if (!milk)
    buy milk;
remove note A
```

Thread B

```
leave note B
if (!noteA){ // Y
    if (!milk)
        buy milk
}
remove note B
```

Can guarantee at X and Y that either:

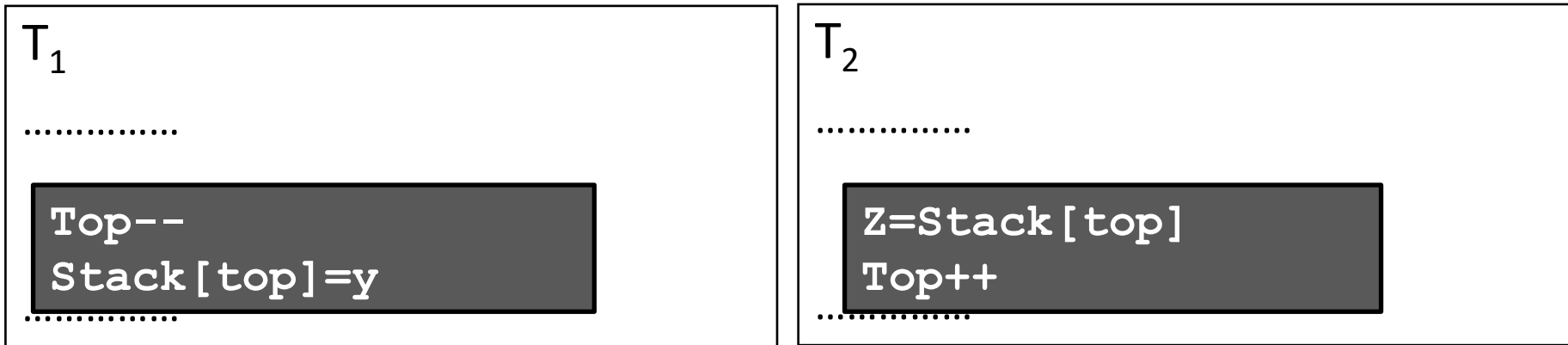
- i) Safe for me to buy
- ii) Other will buy, ok to quit

Lessons

- Solution is complicated
 - “obvious” code often has bugs
 - Demonstrating correctness is not trivial
- Modern compilers/architectures reorder instructions
 - Making reasoning even more difficult
- Generalizing to many threads/processors even more complex
 - Peterson’s algorithm generalization
 - **NOTE:** Peterson’s algorithm is not guaranteed to work on modern processors due to out-of-order execution of instructions. It would require careful insertion of memory barrier instructions (see Lamport’s circular buffer)!

Another example: a stack

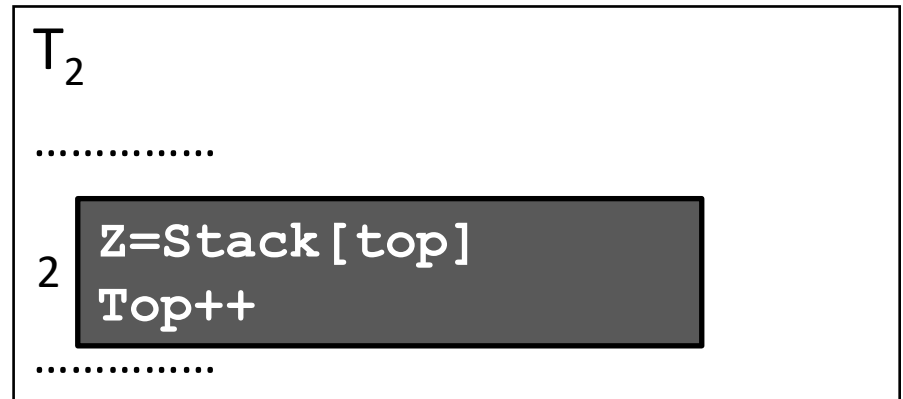
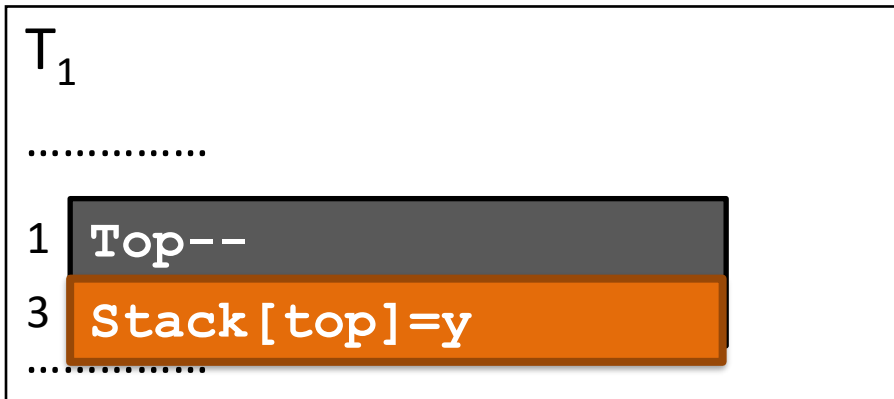
- Two threads interact by means of a stack, with push and pop operations
- Expected behaviour:



 Critical sections

Another example: a stack

- Here the operations of T_1 are interrupted by a pop executed by T_2
- Possible behaviour:



Interference! Race condition.

Another example:

Lamport's circular buffer

- Lamport's Single-Producer Single-Consumer circular buffer uses only load/store instructions (1977). Non-blocking (nbk) algorithm → **busy waiting**

```
nbk_push(data) {  
    if (NEXT(head) == tail)  
        return 0; // buffer full  
    buffer[head] = data;  
    head = NEXT(head);  
    return 1; // OK  
}
```

```
nbk_pop(data) {  
    if (head == tail)  
        return 0; // buffer empty  
    data = buffer[tail];  
    tail = NEXT(tail);  
    return 1; // OK  
}
```

**Doesn't work
on modern
processors!!!**



Non-trivial modifications!

```
nbk_push(data) {  
    if (buffer[tail] != BOTTOM)  
        return 0; // buffer full  
    WMB(); // memory barrier  
    buffer[tail]=data;  
    tail=NEXT(tail);  
    return 1; // OK  
}
```

```
nbk_pop(data) {  
    if (buffer[head] != BOTTOM)  
        return 0; // buffer empty  
    data=buffer[head];  
    buffer[head]=BOTTOM;  
    head=NEXT(head);  
    return 1; // OK  
}
```

**Modified
version for
modern
processors**

BOTTOM is a value that cannot be stored into the buffer.
If we store data pointers into the buffer, *BOTTOM* can be NULL

Lock semantics

- `lock.acquire()`
 - wait until lock is free, then take it
 - C-style syntax, given a lock variable L -> `lock(&L)`
 - `lock.release()`
 - release lock, *waking up anyone waiting for it*
 - C-style syntax, given a lock variable L -> `unlock(&L)`
-
1. At most one lock holder at a time (***safety***)
 2. If no one holding, acquire gets lock (***liveness***)
 3. If lock holder finishes and no higher priority waiters, waiter eventually gets lock (***liveness***)

Too Much Milk, #4

Locks allow concurrent code to be much simpler:

lock.acquire()

if (!milk) buy milk

lock.release()

- How do we implement locks? (discussed later)
 - We need HW support for RMW (Read Modify Write) instructions

Lock Example: Malloc/Free

```
char *malloc (n) {  
    heaplock.acquire();  
    p = allocate memory  
    heaplock.release();  
    return p;  
}
```

```
void free(char *p) {  
    heaplock.acquire();  
    put p back on free list  
    heaplock.release();  
}
```

(*) real implementations of malloc and free are more complex than this example

Rules for Using Locks

- Lock is initially free
- **Always** acquire before accessing shared data structure
 - Beginning of procedure!
- **Always** release after finishing with shared data
 - End of procedure!
 - DO NOT throw lock for someone else to release
- ***Never access shared data without lock***
 - Danger!

Will this code work?

[...]

```
if (p == NULL) {           // No! This check is mistaken! Why?
```

```
    lock.acquire();
```

```
    if (p == NULL) {
```

```
        p = newP();
```

```
    }
```

```
    lock.release();
```

```
}
```

```
use p->field1
```

Where:

```
newP() {
```

```
    p = malloc(sizeof(p));
```

```
    p->field1 = ...
```

```
    p->field2 = ...
```

```
    return p;
```

```
}
```

Lock example: Bounded Buffer

```
tryget() {  
    item = NULL;  
    lock.acquire();  
    if (nelem>0) {  
        item = buf[front];  
        front = (front++)%size;  
        nelem --;  
    }  
    lock.release();  
    return item;  
}
```

```
tryput(item) {  
    r=false;  
    lock.acquire();  
    if (nelem < size) {  
        buf[last] = item;  
        last = (last ++)%size;  
        nelem ++; r=true;  
    }  
    lock.release();  
    return r;  
}
```

Initially: *nelem* = *front* = *last* = 0; *lock* = FREE;
size is buffer capacity

Questions

- If *tryget* returns NULL, do we know the buffer is empty?
 - *No, we only know the buffer **was** empty*
- If we poll *tryget* in a loop, what happens to a thread calling *tryput*?
 - *It could be **delayed** in acquiring the lock (probably it will be)*

Condition Variables

- **Waiting inside a critical section**
 - Synchronization without busy waiting can be achieved using *condition variables and mutex locks*
- Operations: *wait, signal, broadcast*
Called only when holding a lock!
- ***wait***: ***atomically*** releases lock and relinquishes processor until signaled
- ***signal***: wake up **one** waiting thread, if any
- ***broadcast***: wake up **all** waiting threads, if any

Classic problem: Producer-Consumer

Definition:

- One (or more) producer(s) deposits messages in a ***shared buffer***
- One (or more) consumer(s) extracts messages from the shared buffer

Requirements:

- Each produced message must be consumed exactly once.

Variants:

- The shared buffer can have a bounded or unbounded capacity

Example: Bounded Buffer

```
get() {  
    lock.acquire();  
    while (nelem == 0)  
        empty.wait(&lock);  
    item = buf[front];  
    front = (front++) % size;  
    nelem --;  
    full.signal();  
    lock.release();  
    return item;  
}
```

```
put(item) {  
    lock.acquire();  
    while (nelem == size)  
        full.wait(&lock);  
    buf[last] = item;  
    last = (last++) % size;  
    nelem ++;  
    empty.signal();  
    lock.release();  
}
```

Initially: $nelem = front = last = 0$; $size$ is buffer capacity
empty/full are condition variables

Pre/Post Conditions

- What is state of the bounded buffer at lock acquire?
 - $nelem \geq 0$ (and $front \leq last$, without considering the module operation)
 - $nelem \leq size$ (and $front + size \geq last$, without considering the module operation)
 - (also true on return from wait)
- Also true at lock release!
- Allows for proof of correctness

Condition Variable Design Pattern

```
funcThatWaits() {  
    lock.acquire();  
    // Pre-condition: State is consistent  
  
    // Read/write shared state  
  
    while (!testSharedState()) {  
        cv.wait(&lock);  
    }  
    /* WARNING: shared state may have changed  
    (always consistent). We have to test again the  
    condition testSharedState() before going on  
    */  
  
    // Read/write shared state  
    lock.release();  
}
```

```
funcThatSignals() {  
    lock.acquire();  
    // Pre-condition: State is consistent  
  
    // Read/write shared state  
  
    // If testSharedState is now true  
    cv.signal();  
  
    // NO WARNING: signal keeps lock  
  
    // Read/write shared state  
  
    lock.release();  
}
```

Condition Variables

- **ALWAYS** hold lock when calling wait, signal, broadcast
 - Condition variables provide synchronization FOR shared state
 - ALWAYS hold lock when accessing shared state
- Condition variable is **memoryless**
 - If signal when no one is waiting, no op
 - If wait before signal, waiter wakes up
- Wait atomically releases lock
 - What if wait, then release the lock?
 - What if release the lock, then wait?

Condition Variables, cont'd

- When a thread is woken up from wait, it may not run immediately
 - Signal/broadcast put thread on the **Ready list**
 - When lock is released, anyone might acquire it
- Wait **MUST** be in a loop

```
while (needToWait())  
    condition.Wait(lock);
```
- Simplifies implementation
 - Of condition variables and locks
 - Of code that uses condition variables and locks

Java Manual

When waiting upon a Condition, a “*spurious wakeup*” is permitted to occur, in general, as a concession to the underlying platform semantics. This has little practical impact on most application programs as a Condition *should always be waited upon in a loop*, testing the state predicate that is being waited for.

Structured Synchronization

- Identify objects or data structures that can be accessed by multiple threads concurrently
- Add locks to object/module
 - Grab lock on start to every method/procedure
 - Release lock on finish
- If need to wait
 - *while(needToWait()) { condition.Wait(lock); }*
 - Do not assume, when you wake up, that signaller just ran
- If do something that might wake someone up
 - Signal or Broadcast
- Always leave shared state variables in a consistent state
 - When lock is released, or when waiting

Remember the rules

- Use consistent structure
- Always acquire lock at beginning of procedure, release at end
- Always hold lock when using a condition variable
- *Always wait in while loop*
 - *Waiters may be awakened by spurious wake up!*
- Never spin in sleep()

Mesa vs. Hoare semantics

- Mesa (in textbook, Hansen)
 - Signal puts waiter on ready list
 - Signaler keeps lock and processor
- Hoare
 - Signal gives processor and lock to waiter
 - When waiter finishes, processor/lock given back to signaler
 - Nested signals possible!

Mesa & Hoare semantics

- The bounded buffer solution adopted for the Producer-Consumer problem works well with both Mesa & Hoare semantics
- However, it does not make any assumption on the order in which:
 - The producers that are waiting are woken up and deposit their messages
 - The consumers that are waiting are woken up

FIFO Bounded Buffer (Hoare semantics)

```
get() {  
  lock.acquire();  
  while (nelem == 0) // if also works  
    empty.wait(&lock);  
  item = buf[front];  
  front = (front++) % size;  
  nelem --;  
  full.signal(&lock);  
  lock.release();  
  return item;  
}
```

```
put(item) {  
  lock.acquire();  
  while (nelem == size)  
    full.wait(&lock);  
  buf[last] = item;  
  last = (last++) % size;  
  nelem ++;  
  empty.signal(&lock);  
  // CAREFUL: someone else ran  
  lock.release();  
}
```

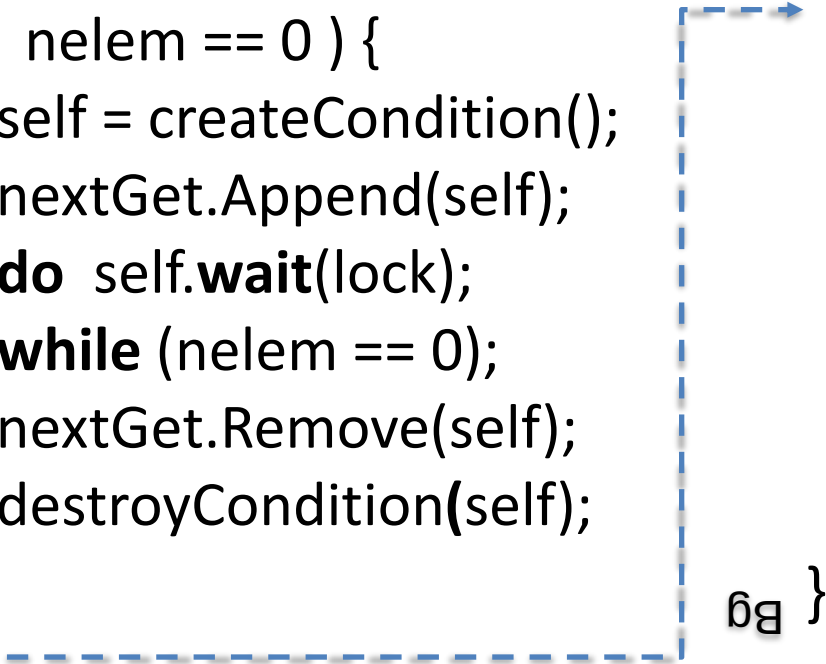
Initially: *nelem* = *front* = *last* = 0; *size* is buffer capacity
empty/full are condition variables

FIFO Bounded Buffer (Mesa semantics)

- Create a condition variable for every waiter
 - Queue condition variables (in FIFO order)
 - Signal picks the front of the queue to wake up
 - CAREFUL if spurious wakeups!
-
- Easily extends to case where queue is LIFO, priority, priority donation, ...
 - With Hoare semantics, not as easy

FIFO Bounded Buffer

(Mesa semantics, put() is similar)

```
get() {  
  lock.acquire();  
  if (!nextGet.empty() ||  
      nelem == 0 ) {  
    self = createCondition();  
    nextGet.Append(self);  
    do self.wait(lock);  
    while (nelem == 0);  
    nextGet.Remove(self);  
    destroyCondition(self);  
  }  
     
}
```

```
item = buf[front];  
front = (front++) % size;  
nelem --;  
if (!nextPut.empty())  
  nextPut.first()->signal();  
lock.release();  
return item;  
}
```

Bg

Initially: *nelem* = *front* = *last* = 0; *size* is buffer capacity
nextGet, *nextPut* are FIFO queues of Condition Variables

Implementing Synchronization

Concurrent Applications

Shared Objects

Bounded Buffer

Barrier

Synchronization Variables

Semaphores

Locks

Condition Variables

Atomic Instructions

Interrupt Disable

Test-and-Set

Hardware

Multiple Processors

Hardware Interrupts

Implementing Synchronization

Take 1: SW implementation using memory
load/store

- See “too much milk” solution / Peterson’s algorithm

Take 2:

```
lock.acquire() { disable interrupts }
```

```
lock.release() { enable interrupts }
```

Take 3: SW implementation with HW support...

Lock Implementation, **Uniprocessor**

```
LockAcquire() {  
    disableInterrupts ();  
    if (value == BUSY) {  
        waiting.add(myTCB);  
        suspend(); *  
    } else {  
        value = BUSY;  
    }  
    enableInterrupts ();  
}  
* Invokes the scheduler,  
  context switch & enables  
  interrupts
```

```
LockRelease() {  
    disableInterrupts ();  
    if (!waiting.Empty()){  
        thTCB = waiting.Remove();  
        readyList.Append(thTCB);  
    } else {  
        value = FREE;  
    }  
    enableInterrupts ();  
}
```



LOCK threw to the
awakened thread!

Multiprocessor

- In multiprocessor, disabling interrupts is not enough
- We need Read-Modify-Write (RMW) instructions
 - Atomically read a value from memory, operate on it, and then write it back to memory
 - Intervening instructions prevented in hardware
- Examples
 - Test-and-set and Compare-and-Swap (CAS) operations
 - Intel: xchgb, lock prefix
 - ARM: LDREX/STREX (“Load-link/Store-conditional”-like ops)
- Does it matter which type of RMW instructions we use?
 - Not for implementing locks and condition variables!

Spinlocks

A Spinlock is a Lock where the processor waits in a loop for the lock to become free (**active waiting!**)

- Assumes lock will be held for a short time
- Used to protect ready list and to implement locks

```
SpinlockAcquire() {  
    while (TestAndSet(&spinLockValue) == BUSY)  
        ;  
}
```

```
SpinlockRelease() {  
    spinLockValue = FREE;  
    memory_barrier();  
}
```

Test-and-Set example

- Let's suppose we have the following *Test-and-Set Lock* (TSL) instruction

```
TSL REG, &ADDR    // REG ← MEMORY[ADDR]  
                  // MEMORY[ADDR] ← #BUSY
```

- The two memory accesses are atomic. Atomicity granted by the HW memory controller.
- TSL is provided in different ways in real processors

Spinlocks implementation with TSL

// &spinLockValue is a memory cell containing a binary value: FREE (0) or BUSY (1)

// TSL R, &spinLockValue :

writes the content of &spinLockValue in R and writes BUSY (1) in
&spinLockValue

spinlockAcquire(&spinLockValue) {

 Loop: TSL R, &spinLockValue

 CMP R, #BUSY

 BEQ Loop // jump if last comparison was successful

 END // at this point &spinLockValue == BUSY!!!!

}

spinlockRelease(&spinLockValue) {

 MOV #FREE, &spinLockValue // this unlocks a thread in the loop, if any

 MFENCE // memory barrier

}

Spinlock implementation in ARM Linux Kernel

```
static inline
void arch_spin_lock(arch_spin_lock *lock){
    unsigned long tmp;
    __asm__ __volatile__(
        "1: ldrex    %0, [%1]\n"
        "    teq     %0, #0\n"
        WFE("ne")
        "    strexeq %0, %2, [%1]\n"
        "    teqeq   %0, #0\n"
        "    bne     1b"
        : "=&r" (tmp)
        : "r" (&lock->lock), "r" (1)
        : "cc");
    smp_mb();
}
```

```
static inline
void arch_spin_unlock(arch_spinlock_t *lock){
    smp_mb();
    __asm__ __volatile__(
        "    str     %1, [%0]\n"
        :
        : "r" (&lock->lock), "r" (0)
        : "cc");
    dsb_sev();
}
```

Spinlock implementation in ARM Linux Kernel

```
static inline
void arch_spin_lock(arch_spin_lock *lock){
    unsigned long tmp;
    __asm__ volatile (
        "1: ldrex %0, [%1]\n"
        "   teq  %0, #0\n"
        "   WFE(\"ne\")\n"
        "   strex %0, %2, [%1]\n"
        "   teqeq %0, #0\n"
        "   bne  1b"
        : "=&r" (tmp)
        : "r" (&lock->lock), "r" (1)
        : "cc");
    smp_mb();
}
```

```
static inline
void arch_spin_unlock(arch_spinlock_t *lock){
    smp_mb();
    __asm__ volatile (
        "   str  %1, [%0]\n"
        :
        : "r" (&lock->lock), "r" (0)
        : "cc");
    dsb_sev();
}
```

Load lock->lock from memory and check whether it is 0. LDREX (*load exclusive*) loads from memory to register and marks memory location as exclusive.

Try to store 1 into lock->lock to acquire the exclusive ownership of the memory location. If the returned value is not 0 we jump again to the LDREX.

If not 0, WFE sets CPU into power saving mode until it receives an interrupt or an event

Memory barrier

Spinlock implementation in ARM Linux Kernel

```
static inline
void arch_spin_lock(arch_spin_lock *lock){
    unsigned long tmp;
    __asm__ __volatile__(
        "1: ldrex %0, [%1]\n"
        "   teq   %0, #0\n"
        WFE("ne")
        "   strexeq %0, %2, [%1]\n"
        "   teqeq %0, #0\n"
        "   bne   1b"
        : "=&r" (tmp)
        : "r" (&lock->lock), "r" (1)
        : "cc");
    smp_mb();
}
```

Memory barrier operation to commit all previous changes in memory

```
static inline
void arch_spin_unlock(arch_spinlock_t *lock){
    smp_mb();
    __asm__ __volatile__(
        "   str   %1, [%0]\n"
        :
        : "r" (&lock->lock), "r" (0)
        : "cc");
    dsb_sev();
}
```

Stores 0 to lock->lock for releasing it

Sends an event to other processors which are waiting at WFE

Lock Implementation, Multiprocessor

```
LockAcquire(){
    disableInterrupts();
    spinLockAcquire(&spinLock);
    if (value == BUSY){
        waiting.add(myTCB);
        sched.suspend(&spinLock);
    } else {
        value = BUSY;
        spinLockRelease(&spinLock);
    }
    enableInterrupts();
}
```

scheduler: marks thread as waiting;
releases spinlock; schedules next thread;

```
LockRelease() {
    disableInterrupts();
    spinLockAcquire(&spinLock);
    if (!waiting.Empty()){
        thTCB = waiting.Remove();
        sched.resume (thTCB,);
    } else value = FREE;
    spinLockRelease(&spinLock);
    enableInterrupts();
}
```

scheduler: marks thread as ready,
puts it in the ready list.

What thread is currently running?

- Thread scheduler needs to find the TCB of the currently running thread
 - To suspend and switch to a new thread
 - To check if the current thread holds a lock before acquiring or releasing it
- On a uniprocessor, easy: just use a global
- On a multiprocessor, various methods:
 - Compiler dedicates a register
 - If hardware has a special per-processor register, use it
 - Fixed-size stacks: put a pointer to the TCB at the bottom of its stack
 - Find it by masking the current stack pointer

Lock Implementation, Linux

- Fast path
 - If lock is FREE, and no one is waiting, then TestAndSet
- Slow path
 - If lock is BUSY or someone is waiting, then see previous slide
- User-level locks
 - Fast path: acquire lock using test&set
 - Slow path: system call to kernel, to use kernel lock

Semaphores

- Semaphore has a non-negative integer value
 - P() atomically waits for value to become > 0 , then decrements
 - V() atomically increments value (if no waiter is present); else it wakes up one waiter
- Semaphore's data structures are integer + queue:
 - Only operations are P and V
 - Operations are atomic
 - If value is 1, two P's will result in value 0 and one waiter
- Semaphores are useful for
 - Unlocked wait: interrupt handler, fork/join

P&V Implementation, Multiprocessor

```
P(sem){  
    disableInterrupts();  
    spinLockAcquire(&spinLock);  
    if (sem.value == 0){  
        waiting.add(myTCB);  
        suspend(&spinLock);  
    } else {  
        sem.value --;  
        spinLockRelease(&spinLock);  
    }  
    enableInterrupts();  
}
```

Invokes scheduler; context switch;
enables interrupts

```
V(sem) {  
    disableInterrupts();  
    spinLockAcquire(&spinLock);  
    if (!waiting.Empty()){  
        thTCB = waiting.Remove();  
        readyList.Append(thTCB);  
    }  
    else sem.value ++;  
    spinLockRelease(&spinLock);  
    enableInterrupts();  
}
```

Throws semaphore to
the awakened thread!

Semaphore Bounded Buffer

```
get() {  
    empty.P();   
    mutex.P();   
    item = buf[front];  
    front = (front + 1) % size;  
    mutex.V();   
    full.V();   
    return item;  
}  
  
put(item) {  
    full.P();   
    mutex.P();   
    buf[last] = item;  
    last = (last + 1) % size;  
    mutex.V();   
    empty.V();   
}
```

The diagram illustrates the semaphore operations for the get() and put(item) functions. Dashed arrows show the flow of semaphore operations: red for empty, blue for full, and green for mutex. In the get() function, empty.P() and full.V() are connected by a red arrow, mutex.P() and mutex.V() by a green arrow, and full.V() is connected to empty.V() in the put(item) function by a blue arrow. The put(item) function also has a green arrow connecting mutex.P() and mutex.V().

Initially: front = last = 0; size is buffer capacity
empty/full are semaphores (initialized to **0** and **size**)
mutex is a semaphore initialized to **1**

Implementing Condition Variables using Semaphores (Take 1)

```
wait(lock) {  
    lock.release();  
    sem.P();  
    lock.acquire();  
}
```

```
signal() {  
    sem.V();  
}
```

Implementing Condition Variables using Semaphores (Take 2)

```
wait(lock) {  
    lock.release();  
    sem.P();  
    lock.acquire();  
}
```

```
signal() {  
    if semaphore not empty  
        sem.V();  
}
```


Implementing Condition Variables using Semaphores (Take 3)

```
wait(lock) {  
    selfsem =  
        createSemaphore();  
    // queue of waiting threads  
    queue.Append(selfsem);  
    lock.release();  
    sem.P();  
    destroySemaphore(selfsem);  
    lock.acquire();  
}  
  
signal() {  
    if !queue.Empty() {  
        seflsem = queue.Remove();  
        selfsem.V();    // wake up waiter  
    }  
}
```

Monitors brief note

- To avoid potential issues related to wrong usage of semaphores and/or locks+CV, the **monitor** concept have been proposed as high-level language construct (e.g., java *synchronized* objects)
- A monitor is an ADT including a set of programmer-defined functions that execute in mutual exclusion within the monitor
- The monitor construct ensures that only one thread at a time is active within the monitor
- A monitor can be implemented with locks+CV or with semaphores
 - From our standpoint monitors are just “*syntactic sugar*” that simplifies concurrent access to a shared object
- In C a monitor can be emulated with a struct with synchronization variables (e.g., lock+CV) and state variables as fields. Then, only a set of well-defined functions operate on the struct’s fields.
 - Consider the *get* and *put* methods of a bounded buffer!

Synchronization Summary

- Use consistent structure
- Always use locks and condition variables (in user-code)
 - “Semaphores considered harmful” Dijkstra, 1968
 - ... still, they are widely used mainly in OSs
- Always acquire lock at beginning of procedure, release at end
- Always hold lock when using a condition variable
- Always wait in while loop
- Never spin in sleep()